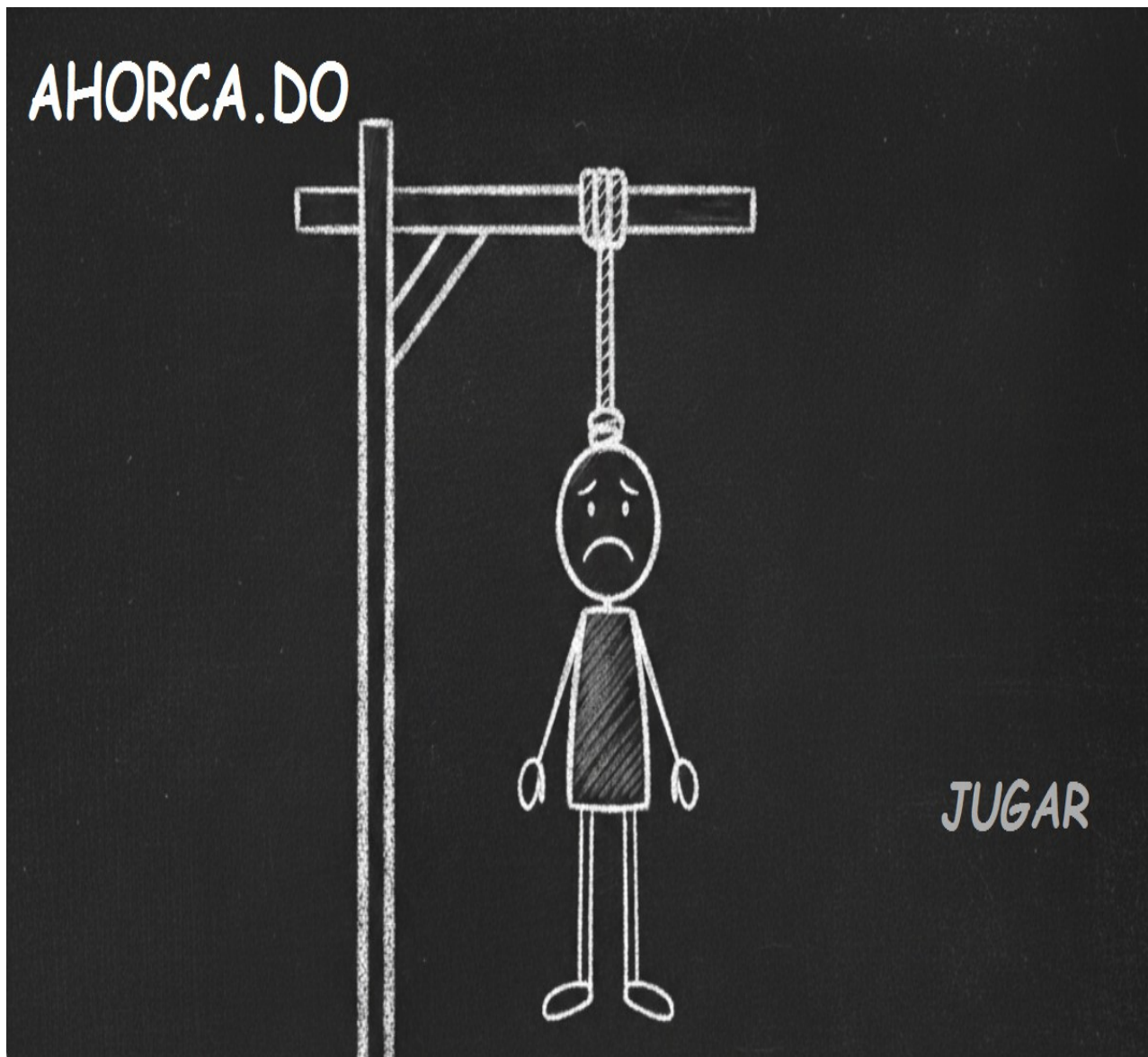


Documentación Técnica: Ahorca.do

Desarrolladores: Bruno Carrodegua Díaz y Nicolás Otero Bouza

Versión: 1.0

Proyecto: “Ahorca_do”



Índice

Documentación Técnica: Ahorca.do.....	1
1. Introducción al Proyecto.....	4
2. Stack Tecnológico.....	4
3. Arquitectura del Sistema.....	4
3.1. Módulo Servidor (ServerSSL).....	5
3.2. Módulo Cliente (ClientTCP & UI).....	5
3.3. Persistencia y Base de Datos (DAO y Modelo).....	5
4. Flujo de Datos y Protocolo de Red.....	6
5. Lógica de Puntuación.....	6
6. Guía de Instalación y Despliegue.....	6
7. Retos Técnicos Superados.....	7

1. Introducción al Proyecto

Ahorca.do es una reimaginación del clásico juego del ahorcado, desarrollado bajo una arquitectura Cliente-Servidor. El proyecto permite a los usuarios registrarse, iniciar sesión de forma segura y jugar tanto en modo solitario como en modo multijugador online mediante un sistema de emparejamiento y turnos.

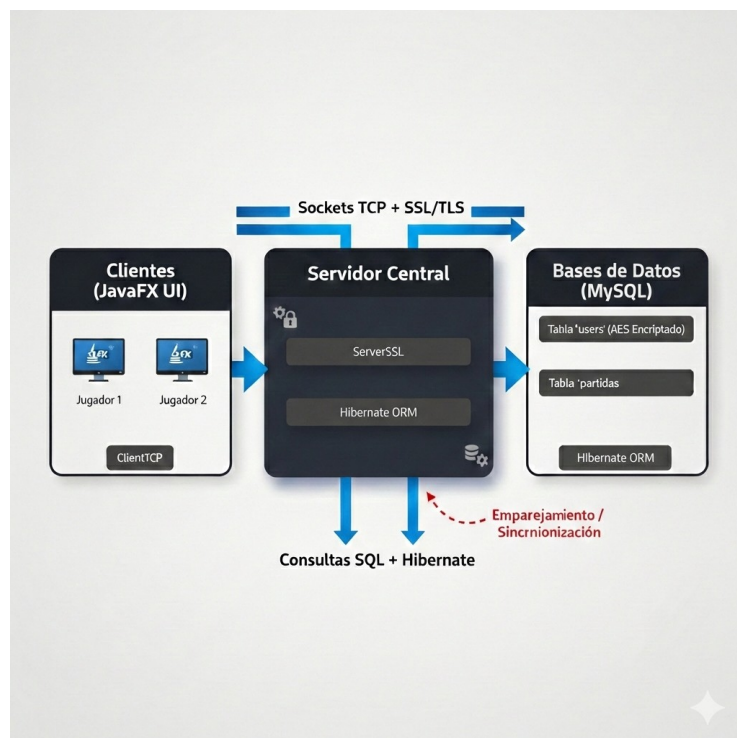
El objetivo principal del desarrollo ha sido implementar un sistema robusto de concurrencia, comunicación en red cifrada y persistencia de datos relacional, todo ello envuelto en una interfaz gráfica amigable desarrollada en JavaFX.

2. Stack Tecnológico

- **Frontend:** JavaFX (Controladores FXML y hojas de estilo CSS personalizadas simulando una pizarra).
- **Backend / Red:** Java Sockets TCP con capa de seguridad SSL/TLS (**SSLSocket**, **KeyStore**).
- **Persistencia de Datos:** Hibernate ORM 7.2 acoplado a una base de datos MySQL 9.1.
- **Seguridad:** * Cifrado de contraseñas en base de datos mediante AES (Advanced Encryption Standard).
 - Certificados JKS (**key.jks**) para la encriptación de los flujos de datos en red.
- **Gestión de dependencias:** Maven (**pom.xml**).
- **Tratamiento de datos:** “**org.json**” para la lectura estructurada del diccionario de palabras.

3. Arquitectura del Sistema

El proyecto sigue un modelo **Ciente-Servidor** estricto donde el cliente es una interfaz "tonta" (solo pinta datos y captura eventos) y el servidor posee la autoridad absoluta sobre la lógica y el estado del juego.



3.1. Módulo Servidor (ServerSSL)

Es el núcleo de la aplicación. Se encarga de:

- **Escucha continua:** Mantiene un **SSLServerSocket** en el puerto 65000.
- **Emparejamiento (Matchmaking):** Si un cliente solicita jugar en Multijugador, se le añade a una **waitingList**. Cuando la lista alcanza dos jugadores, se extraen y se les asigna una instancia compartida de **PlayLogic**.
- **Gestión de Hilos (ThreadClient):** Cada conexión cliente se maneja en un hilo independiente (**Runnable**). Se utiliza sincronización de hilos (**wait()** y **notifyAll()**) para bloquear al jugador que no tiene el turno, evitando condiciones de carrera al procesar las letras.

- **Lógica de Juego (PlayLogic):** Se selecciona una palabra aleatoria leyendo un fichero JSON local. Las vidas se calculan dinámicamente en función de la longitud de la palabra (50% de su longitud, mínimo 1).

3.2. Módulo Cliente (ClientTCP & UI)

La interfaz gráfica utiliza el patrón MVC simplificado:

- **Navegación:** Controlada por el Singleton *AppShell*, que gestiona la carga de archivos .fxml y el intercambio de vistas de forma fluida.
- **Comunicación Asíncrona:** El *GameController* lanza un hilo en segundo plano que escucha constantemente al servidor. Cuando recibe un nuevo estado (**PlayState**), utiliza **Platform.runLater()** para actualizar la UI sin bloquear el hilo principal de JavaFX.
- **Manejo de Errores:** Se capturan silenciosamente excepciones de fin de flujo (**EOFException**) para gestionar desconexiones limpias cuando la partida finaliza o un rival abandona.

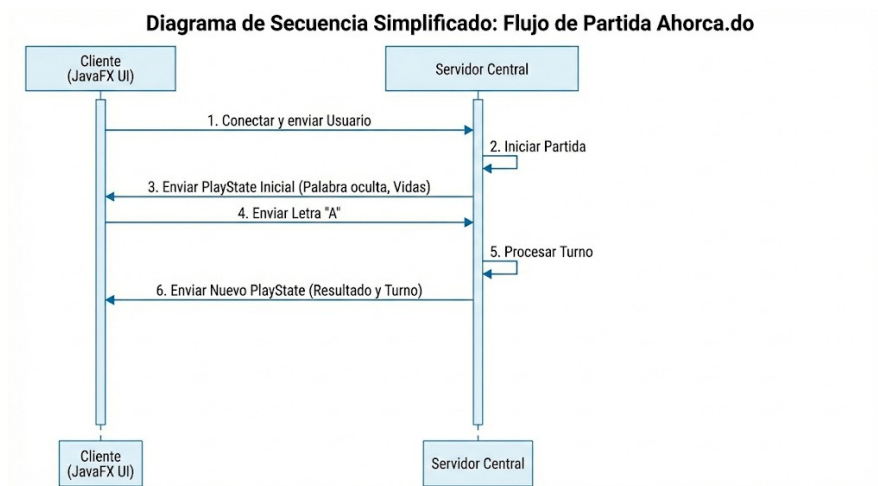
3.3. Persistencia y Base de Datos (DAO y Modelo)

Implementado mediante Hibernate. Consta de dos entidades principales:

- **User:** Almacena credenciales. Incluye un **AttributeConverter (EncryptionConverter)** que intercepta la contraseña antes de hacer el INSERT/UPDATE y la cifra en Base64 usando AES, garantizando que ni siquiera el administrador de la BD pueda leerla en texto plano.
- **Partida:** Registra el historial. Se vincula mediante **@ManyToOne** a los jugadores y al ganador.

4. Flujo de Datos y Protocolo de Red

La comunicación entre Cliente y Servidor se realiza mediante la serialización de objetos (***ObjectOutputStream*** / ***ObjectInputStream***). Todos los modelos transferidos implementan la interfaz ***Serializable***.



Secuencia de inicio de partida:

1. El cliente conecta al Socket SSL y envía un ***Integer*** con el modo de juego (1 o 2).
2. El cliente envía su objeto ***User***.
3. El servidor responde enviando un objeto ***PlayState*** que contiene:
 - La palabra ofuscada (_ _ A _ _).
 - Vidas restantes.
 - Turno actual (***boolean***).
 - Letras falladas (para mostrarlas en el panel lateral).
 - Estado de la partida (Activa/Finalizada).
4. El cliente responde enviando un ***Character*** (la letra pulsada).

5. Lógica de Puntuación

El sistema premia la dificultad. Cuando la lógica del servidor detecta que la palabra ha sido completada (no quedan caracteres _ en el progreso):

- Otorga **1 punto** si la longitud de la palabra secreta es menor a 10 letras.
- Otorga **2 puntos** si la longitud es igual o superior a 10 letras.

El objeto `Partida` se construye en memoria, se le reasignan las entidades "desconectadas" (detached) mediante **`session.merge()`**, y se persiste en la base de datos automáticamente.

6. Guía de Instalación y Despliegue

Para ejecutar el proyecto en un entorno de desarrollo o LAN:

1. Base de Datos:

- Importar o crear un esquema llamado `ahorcado` en MySQL.
- La tabla de usuarios y partidas se generará automáticamente gracias a la propiedad **`hibernate.hbm2ddl.auto = update`**.

2. Configuración de Red (`.properties` y `hibernate.cfg.xml`):

- El Anfitrión (Servidor) debe configurar su IP local (ej. `192.168.x.x`) en ambos archivos.
- Los Clientes deben apuntar esos mismos archivos a la IP del Anfitrión.

3. Ejecución:

- El Anfitrión ejecuta primero la clase **`ServerSSL.java`**.
- Posteriormente, los jugadores ejecutan **`Application.java`** para levantar la interfaz gráfica de JavaFX.

7. Retos Técnicos Superados

Durante el desarrollo surgieron varios desafíos que requirieron rediseños en la arquitectura:

- **Condiciones de carrera en Hibernate:** Solucionado implementando ***session.merge()*** al guardar partidas, ya que los objetos ***User*** que viajaban por el Socket llegaban a la capa DAO en estado *detached*.
- **Bloqueos en la Interfaz (Freeze):** Al principio, la lectura del ***ObjectInputStream*** bloqueaba la ventana. Se solventó derivando la escucha a un ***Thread*** dedicado y volcando las actualizaciones a la UI con ***Platform.runLater()***.
- **Sincronización Multijugador:** El control de turnos requirió el uso de bloqueos condicionales (***wait()*** y ***notifyAll()***) dentro del bloque ***synchronized(play)*** para evitar que el hilo de un jugador consumiera CPU innecesariamente mientras esperaba la jugada de su oponente.